Лабораторная работа №4

Реализация основных моделей в агентном подходе.

Талебу Тенке

Российский университет дружбы народов, Москва, Россия

::: ::::::::::::::
::: ::::::::::::::

Цель работы

Создадим агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-InfectiousRecovered). Модель будет реализована с использованием пакета Agents.jl. В отличие от классической модели на дифференциальных уравнениях, агентный подход позволит учесть индивидуальные характеристики, пространственную структуру и стохастичность процессов.

Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:

- чистый код;
- jupyter notebook;
- документацию в формате Quarto.

6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:

- чистый код;

Выполнение лабораторной работы

Эпидемиологическая модели SIR

- Мы создадим файл src/sir_model.jl. и запустим его.

Ouvrir

sir_model.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab04/project/src

sir_model.jl | sir_run_basic.jl | sir_scan_beta.jl | sir_migration_effect.jl | sir_optimize_parameters.jl

```julia
# Определение типа агента
@agent struct Person(GraphAgent)
    days_infected::Int # количество дней с момента заражения
    status::Symbol # :S, :I, :R
end

# Функция инициализации модели
function initialize_sir(;
    Ns = [1000, 1000, 1000], # численность по трём городам
    migration_rates = nothing, # матрица миграции
    β_und = [0.5, 0.5, 0.5], # передача невыявленными
    β_det = [0.05, 0.05, 0.05], # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1], # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
)
    rng = Xoshiro(seed)
    C = length(Ns) # количество городов

 # Если матрица миграции не задана, создаём реалистичную
if migration_rates === nothing
    migration_rates = zeros(C, C)
    for i = 1:C
        for j = 1:C
            # Вероятность миграции пропорциональна размеру целевого города
            migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
        end
    end
```

Рис. 1: Рис

- Вывод в консоль.

```
julia> include("src/sir_model.jl")
total_count (generic function with 1 method)
```

Рис. 2: Рис

Базовый эксперимент

- Запускаем один эксперимент с фиксированными параметрами (по умолчанию) и сохраняем динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса.

Ouvrir

sir_run_basic.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab04/project/scripts

sir_model.jl | sir_run_basic.jl | sir_scan_beta.jl | sir_migration_effect.jl | sir_optimize_parameters.jl

```julia
# Параметры эксперимента
params = Dict(
    :Ns => [1000, 1000, 1000],
    :β_und => [0.5, 0.5, 0.5],
    :β_det => [0.05, 0.05, 0.05],
    :infection_period => 14,
    :detection_time => 7,
    :death_rate => 0.02,
    :reinfection_probability => 0.1,
    :Is => [0, 0, 1],
    :seed => 42,
    :n_steps => 100,
)

# Инициализация модели
model = initialize_sir(; params...)

# Подготовка массивов для хранения данных
times = Int[]
S_vals = Int[]
I_vals = Int[]
R_vals = Int[]
total_vals = Int[]

# Запуск симуляции вручную
for step = 1:params[:n_steps]
    Agents.step!(model, 1)

    push!(times, step)
    push!(S_vals, susceptible_count(model))
    push!(I_vals, infected_count(model))
    push!(R_vals, recovered_count(model))
```

Базовый эксперимент

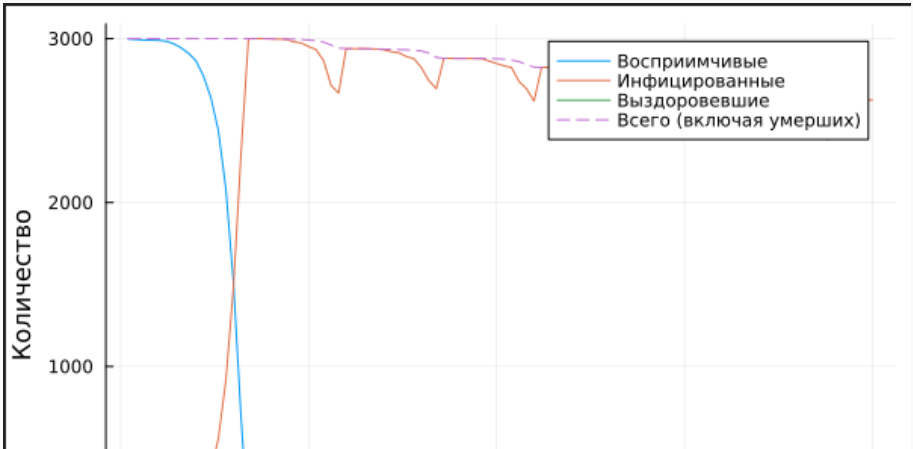
- Мы создадим файл src/sir_run_basic.jl и запустим его.

```
julia> touch("scripts/sir_scan_beta.jl")
"scripts/sir_scan_beta.jl"

julia> include("scripts/sir_scan_beta.jl")
Завершён эксперимент с beta = 0.1, seed = 42
Завершён эксперимент с beta = 0.1, seed = 43
Завершён эксперимент с beta = 0.1, seed = 44
Завершён эксперимент с beta = 0.2, seed = 42
Завершён эксперимент с beta = 0.2, seed = 43
Завершён эксперимент с beta = 0.2, seed = 44
Завершён эксперимент с beta = 0.3, seed = 42
Завершён эксперимент с beta = 0.3, seed = 43
Завершён эксперимент с beta = 0.3, seed = 44
Завершён эксперимент с beta = 0.4, seed = 42
Завершён эксперимент с beta = 0.4, seed = 43
Завершён эксперимент с beta = 0.4, seed = 44
Завершён эксперимент с beta = 0.5, seed = 42
Завершён эксперимент с beta = 0.5, seed = 43
Завершён эксперимент с beta = 0.5, seed = 44
Завершён эксперимент с beta = 0.6, seed = 42
Завершён эксперимент с beta = 0.6, seed = 43
Завершён эксперимент с beta = 0.6, seed = 44
Завершён эксперимент с beta = 0.7, seed = 42
Завершён эксперимент с beta = 0.7, seed = 43
Завершён эксперимент с beta = 0.7, seed = 44
Завершён эксперимент с beta = 0.8, seed = 42
Завершён эксперимент с beta = 0.8, seed = 43
Завершён эксперимент с beta = 0.8, seed = 44
Завершён эксперимент с beta = 0.9, seed = 42
Завершён эксперимент с beta = 0.9, seed = 43
Завершён эксперимент с beta = 0.9, seed = 44
```

Базовый эксперимент

- График: plots/sir_basic_dynamics.png — четыре линии: S(t), I(t), R(t) и общая численность. Позволяет визуально оценить пик эпидемии, скорость распространения и влияние смертности.



Сканирование коэффициента заразности

- Исследуем, как изменение базовой заразности (β_und и пропорционально β_det) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности.



Сканирование коэффициента заразности

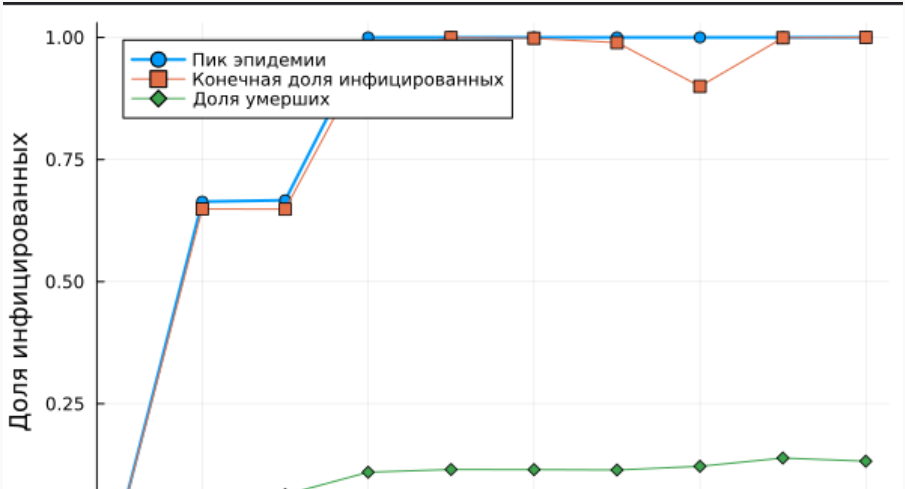
- Создание файла scripts/sir_scan_beta.jl и запустим его.

```
julia> touch("scripts/sir_migration_effect.jl")
"scripts/sir_migration_effect.jl"

julia> include("scripts/sir_migration_effect.jl")
Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44
Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
```

Сканирование коэффициента заразности

- График позволяет найти пороговое значение $?$, при котором возникает эпидемия, и оценить, как увеличивается нагрузка на систему с ростом заразности



Исследование эффекта миграции

- Исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы.

Ouvrir

sir_migration_effect.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab04/project/scripts

sir_model.jl | sir_run_basic.jl | sir_scan_beta.jl | sir_migration_effect.jl | sir_optimize_parameters.jl

```julia
# Функция для создания матрицы миграции с заданной интенсивностью
function create_migration_matrix(C, intensity)
M = ones(C, C) .* intensity ./ (C-1)
for i = 1:C
M[i, i] = 1 - intensity
end
return M
end

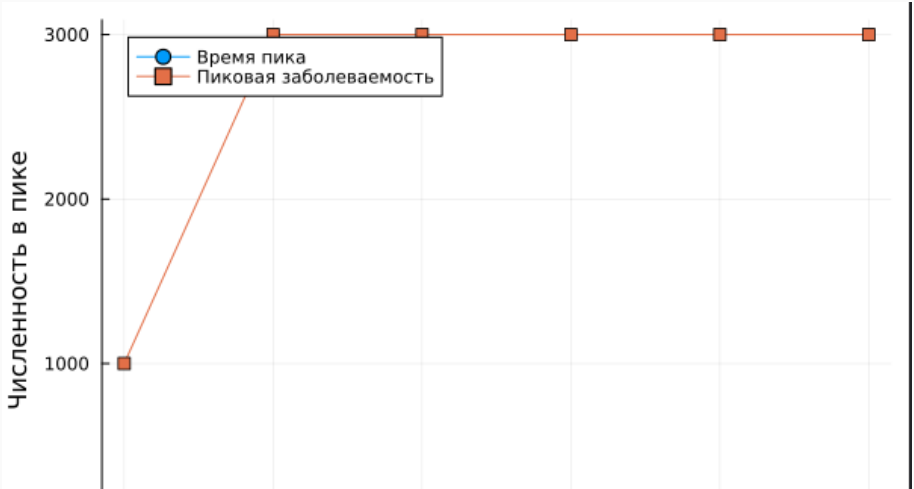
# Функция для измерения времени достижения пика
function peak_time(p)
    # Создаём матрицу миграции на основе интенсивности
    migration_rates = create_migration_matrix(p[:C], p[:migration_intensity])
    model = initialize_sir(;
        Ns = p[:Ns],
        β_und = p[:β_und],
        β_det = p[:β_det],
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        migration_rates = migration_rates,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(model)
    peak = 0.0
    peak_step = 0

    for step = 1:p[:n_steps]
        # Ручной шаг
        agent_ids = collect(allids(model))
```

Исследование эффекта миграции

- График демонстрирует, как ускорение обмена людьми между городами приводит к более раннему и более высокому пику эпидемии.



Многокритериальная оптимизация параметров

- Ищим оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Используем эволюционный алгоритм (Borg MOEA) из пакета BlackBoxOptim.



- Создание файла scripts/sir_optimize_parameters.jl и запустим его.

```
Optimization stopped after 1 steps and 889.28 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.06
Improvements/step = NaN
Total function evaluations = 51


Best candidate found: [0.122956, 5.34903, 0.0237749]

Fitness: (0.00067, 0.00000) agg=0.00067

Оптимальные параметры:
β_und = 0.12295579898158256
Время выявления = 5 дней
Смертность = 0.023774934501594132
Достигнутые показатели:
```

Многокритериальная оптимизация параметров

- Вывод в консоль: оптимальные параметры и соответствующие им значения критериев.

Сводная визуализация результатов

- Этот скрипт загружает результаты параметрического сканирования (файл data/beta_scan_all.csv, созданный scan_beta.jl) и строит единый составной график, объединяющий три панели:

— пик эпидемии и конечная доля инфицированных; — число умерших; — доля выздоровевших.

Сводная визуализация результатов

График сохраняется как plots/comprehensive_analysis.png. Скрипт предназначен для итогового отчёта и не требует повторного запуска симуляций.



Рис. 13: Рис

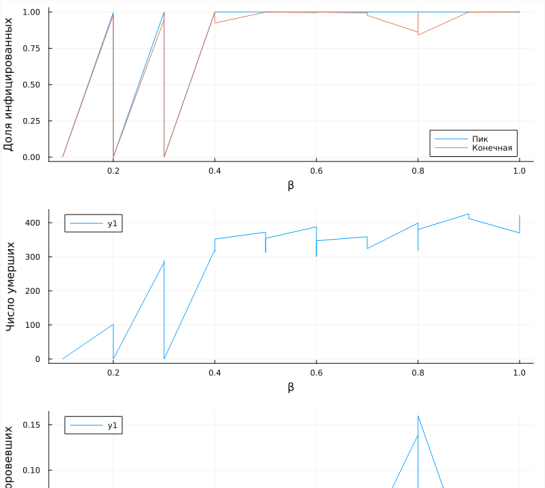
Сводная визуализация результатов

- График: plots/comprehensive_analysis.png (рис. 4.4). Три панели, позволяющие одновременно оценить:

— порог возникновения эпидемии (рост пика); — нелинейное увеличение смертности; — насыщение доли переболевших.

Сводная визуализация результатов

- В консоль может выводиться дополнительная статистика (например, значения β, при которых пик превышает 5%).



Выводы

- Я выполнила лабораторную работу.